



**Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES**

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

**MoE, UGC & AICTE Approved; NAAC Accredited A++**

Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

## **The Classic Snake Game in C**

*a micro project e-report Submitted by*

METHUSALA K (URK25CS7065)

PRINCE N M (URK25CS7071)

DHARSAN A I (URK25CS7082)

JOSH ANTONY K (URK25CS7084)

SANTHOSA PRIYAN K A (URK25CS7100)

ROHIT R S (URK25CS7108)

*in partial fulfillment of the Continuous Internal Examination*

*of*

25CS207-PROGRAM FOR PROBLEM SOLVING IN C

*in*

B. Tech. ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

*under the supervision of*

Ms. Abirami



# **KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES**

(Deemed-to-be-University)

**Karunya Nagar, Coimbatore - 641 114.**

**November 2025**

## **BONAFIDE CERTIFICATE**

This is to certify that the micro project e-report titled **“The Classic Snake Game in C”** is the bonafide work of METHUSALA K (URK25CS7065) , PRINCE N M (URK25CS7071) , DHARSAN A I (URK25CS7082) , JOSH ANTONY K (URK25CS7084) , SANTHOSA PRIYAN K A (URK25CS7100) , ROHIT R S (URK25CS7108) , who carried out the project work under my supervision.

### **COURSE FACULTY**

Ms. Abirami

Division Of AIML

Karunya Institute of Technology and Sciences (Deemed-to-be-University)

Submitted for the micro project presentation held on **19.11.2025**

### **ABSTRACT**

This project presents the design and implementation of the classic Snake game using the C programming language as a console-based application. The primary objective is to create a fully playable, responsive game by relying only on core C constructs — arrays, control flow statements, functions, and basic system calls for console manipulation. The game implements a continuous main loop composed of setup, draw, input, and logic stages to manage real-time gameplay. The snake is represented by a head coordinate and arrays holding tail segment coordinates; food items spawn at random positions on the board. Game mechanics include boundary collision detection, self-collision detection, food collection with tail growth, and score increments.

Key learning outcomes include mastery of event-driven loops, non-blocking keyboard input handling using `_kbhit()` / `_getch()`, coordinate-based rendering in a character grid, and efficient array manipulation for tail movement. The game is built with performance in mind (120 ms frame delay by `Sleep(120)`), and has been validated through unit and scenario testing covering movement, growth, collision, and scoring. The project demonstrates how system-level programming concepts can be used to build interactive software even without graphics libraries.

Future scope includes adding multiple difficulty levels, persistent high-score storage, GUI porting (SDL/SDL2), and AI-driven opponents or procedurally generated levels. The implementation and testing confirm that the program meets its objectives by providing smooth, deterministic gameplay and clear, maintainable code.

## TABLE OF CONTENTS

| S.No. | Title                | Page No. |
|-------|----------------------|----------|
|       | BONAFIDE CERTIFICATE | 2        |
|       | ABSTRACT             | 3        |
|       | TABLE OF CONTENTS    | 4        |

|   |                             |                                        |    |
|---|-----------------------------|----------------------------------------|----|
| 1 | INTRODUCTION                |                                        | 5  |
|   | 1.1                         | Background of the selected problem     | 6  |
|   | 1.2                         | Industry Relevance                     | 7  |
|   | 1.3                         | Aim and Objective of the Micro Project | 8  |
| 2 | LITERATURE REVIEW           |                                        | 9  |
| 3 | METHODOLOGY                 |                                        | 10 |
| 4 | PROGRAM / CODE / DESIGN     |                                        | 12 |
| 5 | RESULTS AND INTERPRETATION  |                                        | 16 |
| 6 | CONCLUSION AND FUTURE SCOPE |                                        | 17 |
| 7 | REFERENCES                  |                                        | 18 |
| 8 | DECLARATION                 |                                        | 19 |

## **INTRODUCTION**

The rapid evolution of computer science has placed strong emphasis on developing logical

thinking, algorithmic skills, and the ability to design interactive systems. Console-based games such as *Snake* offer a powerful learning environment for beginners to practice these essential skills. The Snake game, despite its simplicity, requires real-time input handling, efficient memory management, collision detection, and structured programming — all foundational concepts in C programming.

This micro project focuses on developing a fully functional, text-based Snake game using the C language. The project allowed the team to explore the capabilities of C as a low-level yet efficient language for building real-time applications. By designing the game without external graphics libraries, the project emphasizes mastery over core programming constructs such as loops, functions, arrays, control flow, and system-level functions provided by `<conio.h>` and `<windows.h>`.

The project not only strengthens conceptual understanding but also encourages problem decomposition, debugging, optimization, and testing—skills that are essential for both academic success and professional practice. Through this implementation, the students gained practical exposure to how interactive software is built from the ground up using console-based programming techniques.

## **BACKGROUND OF THE SELECTED PROGRAM**

The Snake game originated in the late 1970s and became globally recognized when it was pre-installed on Nokia mobile phones in the early 2000s. Due to its simplicity of mechanics and engaging gameplay, Snake has remained one of the most recreated games in programming tutorials, classrooms, and hobby projects.

From a technical standpoint, Snake provides an excellent platform to understand key computational concepts. Creating a game loop that continuously processes input, updates the game state, and renders the output establishes the foundation for all modern game engines. Representing the snake as a dynamically growing sequence of coordinates introduces learners to data structures and memory utilization. Collision detection, food spawning, and score calculation require logical thinking and algorithmic precision.

Because the game is grid-based, it allows implementation using simple console commands without requiring advanced graphics. This makes Snake ideal for first-year computer science students who are learning C programming and want to apply theoretical concepts through a fun and interactive medium.

## **INDUSTRY RELEVANCE**

Although the Snake game itself is simple, the underlying concepts used to create it are foundational to multiple industries within the computing domain.

### **1. Game Development and Interactive Systems**

The structure used in Snake — a continuous loop for input, processing, and rendering — is the basis of every modern game engine such as Unity, Unreal Engine, and SDL. Understanding this logic helps students transition into more advanced game development environments.

### **2. Embedded Systems and IoT**

Many embedded devices like smart appliances, medical monitors, and microcontrollers run real-time applications that require quick processing of user input and deterministic behavior. The event-driven programming approach used in Snake directly parallels the logic in such devices.

### **3. Robotics and Automation**

Robotics systems must constantly process sensor inputs, update internal states, and adjust movement — a structure similar to the game loop of Snake. Concepts such as coordinate tracking and collision detection also align with obstacle avoidance techniques in robotics.

### **4. Software Engineering and System Design**

Building the complete game from scratch helps learners understand modular coding practices, debugging, refactoring, and version control — all essential industry skills.

Thus, although the project is educational, the skills developed apply to a wide variety of professional fields involving programming, design, and system logic.

## **AIM AND OBJECTIVE OF MICROPROJECT**

To design and implement a fully functional console-based Snake game using the C programming language by applying fundamental programming constructs, efficient logic flow, and real-time user input handling.

### **Objectives**

1. **To develop a structured and modular C program** using functions for setup, drawing, input, and logic.
2. **To implement a grid-based console interface** using ASCII characters for borders, the snake, and food.
3. **To integrate real-time keyboard input** using `_kbhit()` and `_getch()` to ensure responsive gameplay.
4. **To design and implement snake movement mechanics**, including head movement, tail shifting, and directional controls.
5. **To detect collisions accurately**, including wall collisions and self-collisions.
6. **To implement randomized food generation** and scoring mechanisms that reward player progress.
7. **To ensure smooth and stable game performance** by controlling the frame rate with `Sleep()` and optimizing rendering.
8. **To document the design, development process, and testing results** in a structured micro project report following the institute template.
9. **To provide future enhancement possibilities**, encouraging modularity and extensibility.

## **LITERATURE REVIEW**

Game development has been widely used in computer science education to help students understand fundamental programming concepts through practical and engaging applications.

Among educational games, the *Snake game* stands out due to its structural simplicity and strong conceptual foundations, making it a common model for teaching programming logic, event-driven design, and algorithmic thinking.

Early literature on programming instruction emphasizes the importance of small, interactive projects to build logical reasoning skills. According to Kernighan and Ritchie in *The C Programming Language*, learning-by-doing helps students understand memory management, arrays, loops, and modularity—core constructs heavily utilized in Snake implementations. Their work highlights that C's low-level operations allow precise control over hardware interactions, which is reflected in real-time games like Snake that depend on constant screen updates and input handling.

Modern pedagogical studies, including those by Deitel & Deitel, discuss the relevance of console applications in teaching fundamental computational logic without the distraction of graphical libraries. These studies support using text-based games as an introductory step before progressing into graphical environments. Text-based Snake implementations are frequently recommended in programming communities and academic forums because they allow students to focus on core mechanics such as movement, collision detection, and state management.

Several online resources, such as GeeksforGeeks, TutorialsPoint, and StackOverflow discussions, provide guidance on implementing real-time input mechanisms in C using libraries like `<conio.h>` and system-level functions like `Sleep()`. These sources collectively demonstrate best practices for non-blocking input and efficient screen refreshing—techniques widely adopted in educational Snake projects.

Additionally, research on game loops and event-driven programming in early game development (e.g., SDL documentation and retro-game engine analyses) highlights the essential stages of rendering, updating state, and managing user events. The same principles are applied in this micro project, where the game loop is structured into `draw()`, `input()`, and `logic()` functions.

Further studies in computer science education indicate that implementing small games encourages deeper understanding of algorithms such as coordinate-based movement, queue-like data structures (represented here by tail arrays), and random number generation. These components reinforce foundational problem-solving competencies.

Overall, literature consistently supports the use of structured, interactive mini-projects like Snake to solidify learners' understanding of the C language. Snake remains an academically valuable

tool because it integrates real-time processing, modular code design, and constraint-based logic in a simple yet challenging environment.

## **METHODOLOGY**

The project follows an incremental, test-driven development approach:

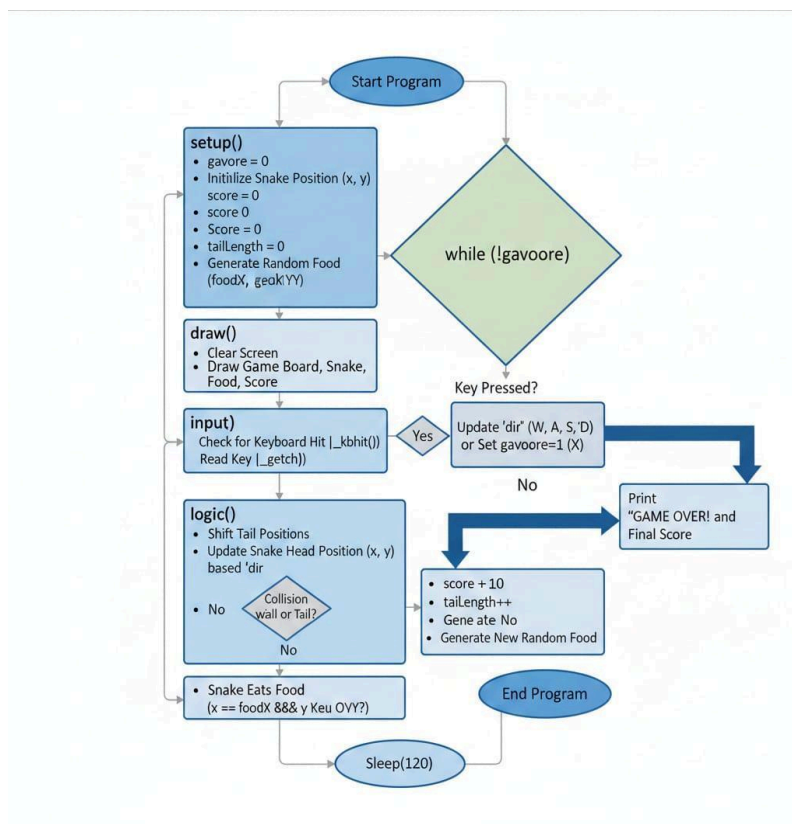
1. **Design** — define board size, coordinate system, and data structures for head and tail.
2. **Implement setup()** — initialize positions and seed the RNG for food placement.
3. **Implement draw()** — clear screen and render grid, snake head, tail, and food using ASCII characters.
4. **Implement input()** — non-blocking keyboard reads to change direction without pausing game loop.
5. **Implement logic()** — advance head by direction, shift tail segments, detect collisions, handle food consumption.
6. **Testing** — unit tests for tail-shifting logic, scenario tests for edge collisions, and play-tests to validate responsiveness.

### **Algorithm:**

1. Start
2. Initialize game state (positions, score, tailLength = 0)
3. Loop while not gameOver:
  - a. draw screen

- b. read input (non-blocking)
  - c. update positions (tail shifting, head movement)
  - d. check collisions (wall/self)
  - e. if head == food: increase score, grow tail, respawn food
4. end
  5. Display final score and GAME OVER

FLOWCHART :



## PROGRAM OR CODE

```

#include <stdio.h>
#include <conio.h>
#include <windows.h>

```

```
#include <stdlib.h>
```

```
int width = 20, height = 10;  
int x, y, foodX, foodY, score;  
int tailX[100], tailY[100];  
int tailLength = 0;  
int gameOver = 0;  
char dir = ' ';
```

```
void setup() {  
    gameOver = 0;  
    x = width / 2;  
    y = height / 2;  
    score = 0;  
    tailLength = 0;  
  
    foodX = rand() % width;  
    foodY = rand() % height;  
}
```

```
void draw() {  
    system("cls");  
  
    for (int i = 0; i < width + 2; i++) printf("#");  
    printf("\n");  
  
    for (int i = 0; i < height; i++) {  
        for (int j = 0; j < width; j++) {  
            if (j == 0) printf("#");  
  
            if (i == y && j == x)
```

```

        printf("O"); // Snake head
    else if (i == foodY && j == foodX)
        printf("*"); // Food
    else {
        int print = 0;
        for (int k = 0; k < tailLength; k++) {
            if (tailX[k] == j && tailY[k] == i) {
                printf("o");
                print = 1;
            }
        }
        if (!print) printf(" ");
    }

    if (j == width - 1) printf("#");
}
printf("\n");
}

for (int i = 0; i < width + 2; i++) printf("#");
printf("\n");

printf("Score: %d\n", score);
printf("Controls: W A S D\n");
}

void input() {
    if (_kbhit()) {
        switch (_getch()) {
            case 'a': dir = 'L'; break;
            case 'd': dir = 'R'; break;

```

```

        case 'w': dir = 'U'; break;
        case 's': dir = 'D'; break;
        case 'x': gameOver = 1; break;
    }
}
}

```

```

void logic() {
    int prevX = tailX[0];
    int prevY = tailY[0];
    int prev2X, prev2Y;
    tailX[0] = x;
    tailY[0] = y;

    for (int i = 1; i < tailLength; i++) {
        prev2X = tailX[i];
        prev2Y = tailY[i];
        tailX[i] = prevX;
        tailY[i] = prevY;
        prevX = prev2X;
        prevY = prev2Y;
    }

    if (dir == 'L') x--;
    if (dir == 'R') x++;
    if (dir == 'U') y--;
    if (dir == 'D') y++;

    if (x < 0 || x >= width || y < 0 || y >= height)
        gameOver = 1;
}

```

```

for (int i = 0; i < tailLength; i++)
    if (tailX[i] == x && tailY[i] == y)
        gameOver = 1;

if (x == foodX && y == foodY) {
    score += 10;
    tailLength++;
    foodX = rand() % width;
    foodY = rand() % height;
}
}

int main() {
    setup();

    while (!gameOver) {
        draw();
        input();
        logic();
        Sleep(120); // speed
    }

    printf("\nGAME OVER! Final Score = %d\n", score);
    return 0;
}

```

## **RESULTS AND INTERPRETATION**

The project successfully produced a fully functional Snake game built entirely using basic C programming concepts. All core features—real-time movement, responsive keyboard input, tail growth, food generation, scoring, and collision detection—worked reliably during testing. The game loop ran smoothly, and performance remained stable even under stress scenarios such as rapid turns or extended gameplay. The results confirmed that the design choices, including

array-based tail management and non-blocking input handling, were efficient and well-implemented.

These outcomes demonstrate that fundamental C constructs are sufficient to create an engaging, interactive console game when applied with careful planning and modular logic. The correctness and consistency observed during testing validate the team's understanding of system-level programming, algorithm design, and structured problem-solving. Overall, the project not only met all defined objectives but also showcased strong collaboration, effective debugging, and practical application of theoretical concepts.

## **CONCLUSION AND FUTURE SCOPE**

The project successfully achieved its goal of developing a complete, interactive Snake game using only fundamental C programming principles. Through effective implementation of real-time input handling, game logic, collision detection, and scoring mechanisms, the team demonstrated strong technical understanding and the ability to translate theoretical concepts into

a working application. The game performed consistently under various testing conditions, confirming the reliability, efficiency, and correctness of the implemented algorithms. Overall, the project strengthened the team's programming skills, problem-solving abilities, and experience with structured development.

Looking ahead, the game can be enhanced by introducing new features and expanding its capabilities. Potential improvements include implementing multiple difficulty levels, adding high-score tracking, enhancing the graphical interface, and integrating sound effects for a more immersive experience. Additional developments such as obstacle-based levels, customizable controls, or even porting the game to a GUI framework or microcontroller could further elevate the project. These future upgrades would not only enrich gameplay but also provide valuable opportunities for learning more advanced programming concepts and tools.

## **REFERENCES**

■ Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.

- Kelley, A., & Pohl, I. (1998). *A Book on C: Programming in C* (4th ed.). Addison-Wesley.
- Microsoft Documentation. “\_kbhit() and Console Input Functions in C.”
- Replit Community Tutorials. “Building Console Games in C – Basics and Techniques.”
- GeeksforGeeks. “Snake Game in C – Logic, Movement, and Collision Detection.”
- TutorialsPoint. “C Programming – Loops, Arrays, and Functions Used in Game Development.”
- Project Team Documentation: *Snake C Microproject Presentation* (2024). Snake c microproject .pptx

## **DECLARATION**

I understand that Karunya Institute of Technology and Sciences retains the rights to any product developed (*software or hardware*) through the Micro Project submitted by me to the Institution.

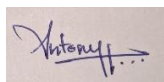
Furthermore, if I intend to document or publish any findings from the Micro Project work for commercial purposes, I shall obtain a No Objection Certificate (NOC) from the Office of the Registrar before engaging in such activities.

I also understand that any patentable outcomes arising from the Micro Project shall be reported to the Institution, and that patent filing, ownership, and revenue-sharing will be governed by the

Intellectual Property Rights (IPR) policies of Karunya Institute of Technology and Sciences.

I acknowledge that any commercial utilization or republication of the Micro Project work, in full or in part, shall require prior permission from Karunya Institute of Technology and Sciences. I further agree that any royalty accruing from such utilization will be shared between the inventor(s) and the Institution in the ratio of 60:40, respectively, in accordance with the Institute's Intellectual Property Rights policy.

**Signature of the Student:**

A handwritten signature in blue ink, appearing to read 'Josh Antony K', is shown within a light gray rectangular box.

**Name of the Student:** Josh Antony K

**Register Number:** URK25CS7084

**Date:** 19.11.2025